

Harrish_VP

August 15, 2026

DATA ENGINEERING ASSESSMENT- ORIGIN MEDICAL

TASK - 1

MOTIVATION : -

Ever since I've started learning in this field of Machine Learning, I was interested in aspects beyond the field itself. Questions like how can we apply this knowledge, paradigms, etc, to an entirely different field, be it Medical Imaging, finance, agriculture, or even be robotics, efficiently. For me, our work always needs to have an impact and scope beyond ourselves, and this is what I've always strived to work and converge at. Medicine is my answer to this question, and it's one reason why I'm very interested in this medical-AI field and especially, Origin Medical: The ability to have an explicit impact on people's lives.

I understand that this is supposed to be strictly about the take home assessment, but please feel free to take a look at my current research paper (https://anonymous.4open.science/r/SAML-ra-D151/SAML_2nd_paper.pdf). This paper serves as a definitive proof of my knowledge within Medical Imaging and it's pipelines. I've researched on this domain extensively, and I understand how critical patient data and regulatory compliance matters, more than a general data engineer.

For this task, I wanted to look at the segmentation dataset from that perspective. Rather than directly trying to train a U-Net or compare architectures, the idea was to first understand what the images look like, how the annotations behave, and whether there is enough useful variation in the dataset to justify training an AI model on it.

I also found this task interesting because the dataset is relatively small. With only a limited number of samples, understanding where the variation actually comes from becomes important. If the images are all very similar, the model may learn the acquisition setup rather than the anatomy itself. For this reason, I focused the analysis around image consistency, annotation geometry, spatial distribution and some basic quality checks.

INTRODUCTION ABOUT THE TASK : -

The assigned task was to design and implement a containerized batch pipeline which accepts healthcare data in two forms: DICOM images and PDF reports. The pipeline is expected to de-identify patient information, upload the sanitized files to target storage, extract useful metadata and provide some level of observability around what happened to each file.

I split the implementation into two preprocessing paths, one for DICOM and one for PDF, and kept the storage and metadata layers shared. This makes the source-specific work stay where it belongs, while the rest of the pipeline does not need to know whether a file came from DICOM or PDF.

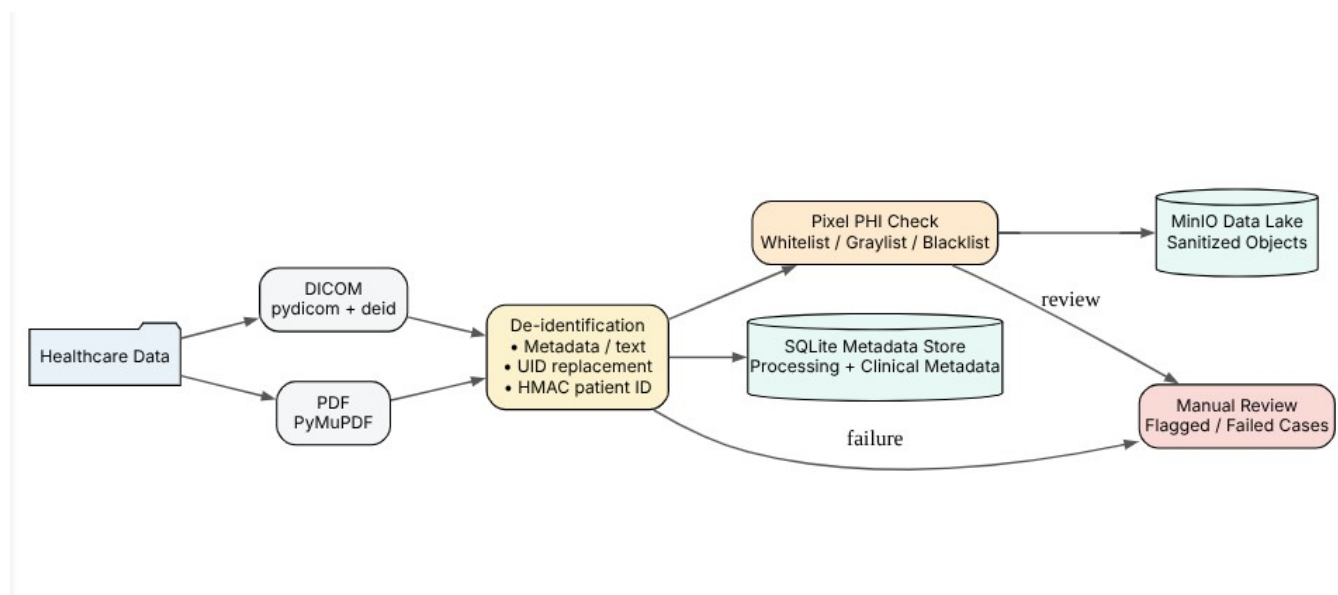


Figure 1. High-level flow of the implemented pipeline. (generated via GraphViz)

For multiple developers, I would use short-lived feature branches from a protected main branch. Each change would be pushed through a Pull Request, reviewed and checked by CI before being merged. I would avoid directly committing to main, especially for changes around de-identification rules or storage behaviour, since a small mistake there can have a much larger effect than a normal feature bug.

DATA EXTRACTION, PREPROCESSES, AND ANALYSIS : -

3.1 DICOM extraction and de-identification : -

The DICOM path starts by reading the file metadata with pydicom before the actual de-identification step. The implementation uses the deid library with separate de-identification recipes, and private fields are removed during replacement. Patient identifiers are converted into a deterministic HMAC based value before being used internally. The raw PatientID is therefore not carried through as the database identifier or storage name.

For the UID fields, I did not use a completely random replacement. Study, Series, SOP and Frame of Reference UIDs are regenerated deterministically from the original UID and a namespace using the same HMAC secret. This keeps the identifiers stable across repeated processing while still removing the original value.

The pixel data is checked separately using DicomCleaner. If a graylist rule contains coordinates that can actually be cleaned, the pipeline applies the pixel cleanup and writes the cleaned DICOM back. Whitelist and blacklist matches are recorded differently, while graylist cases without a cleanable region are sent to a review state. I wanted the pixel step to be slightly more careful than simply assuming that metadata de-identification means the whole file is safe.

3.2 PDF extraction and de-identification: -

For PDF reports, I used PyMuPDF to extract words and locate specific patient fields. The current parser looks for Patient ID, Patient Name, Gender, Patient Age, GA and BMI. Patient ID is converted to the same HMAC based identifier used by the DICOM path, while the patient name is replaced with REDACTED.

The patient ID and name are also redacted from the actual PDF page content using the located word coordinates. After the redaction is applied, the document metadata is cleared before the sanitized PDF is saved. If a Patient ID cannot be found, the file is skipped and logged instead of being stored without an identifier.

3.3 Storage and Metadata : -

Sanitized files are uploaded to a MinIO based data lake. The storage layer is intentionally kept behind a small DataLake class, so the preprocessing code only needs to call `put()` and does not need to know about the underlying client implementation. The bucket is checked when the storage object is initialized and is created if it is not already present.

The metadata database is SQLite for this implementation. Separate tables are created for DICOM and PDF metadata. DICOM records store the pseudonymous patient ID, modality, regenerated UUIDs, manufacturer information, body part, dimensions, pixel spacing, slice thickness, patient sex/age, pixel status, processing status and object storage key. PDF records store the pseudonymous patient ID and extracted report fields such as gender, age, gestational age and BMI along with the storage key.

`INSERT OR REPLACE` is used for metadata writes, which also gives the DICOM path a simple idempotent behaviour when the same file is processed again and the generated identifiers remain deterministic.

3.4 Configuration and observability: -

Configuration is loaded from YAML, while the HMAC secret and MinIO credentials are taken from environment variables. This keeps secrets out of the source code and keeps the pipeline paths configurable without changing the processing logic.

Logging is handled through a shared logger with timestamp, level, module name and message. The batch loop catches file-level exceptions, logs the exception and continues with the remaining files. The final batch log records processed and failed counts, while DICOM records also retain a pixel status and processing status for later inspection.

3.5 3.5 Execution flow : -

The main entry point creates the shared DataLake and MetadataStore objects and then runs the DICOM batch. The PDF path is implemented as a separate function and follows the same storage and metadata route, but it is currently commented out in `main`. I kept this as a staged execution point rather than silently implying that both paths are enabled in the default run.

Pseudo-code: -

```
load config + environment variables

connect to metadata store + object storage

for each DICOM file:
    de-identify -> regenerate IDs -> inspect pixels ->
upload -> store metadata

for each PDF file:
    extract -> redact -> pseudonymize -> upload -> store
metadata

catch file errors -> log -> continue batch -> report counts
```

.

3.6 Design decisions and trade-offs : -

- **SQLite for metadata:** Simple and lightweight for the current workload, but concurrent writes and horizontal scaling are limited.
- **MinIO for file storage:** Keeps DICOM/PDF objects out of the database and makes storage easier to manage. The trade-off is keeping metadata and object storage in sync.
- **Deterministic HMAC IDs:** Keeps patient identifiers stable across reruns without exposing the original ID. The trade-off is that the same secret must be protected since it controls the mapping.
- **Per-file failure handling:** A bad file does not kill the whole batch. The trade-off is that failed records have to be reviewed separately after the run.
- **Rule-based PDF extraction:** Kept the initial implementation simple and predictable for the expected report format. The trade-off is that it will not generalise as well to arbitrary PDF layouts, which is where the proposed local NLP model would come in.

RESULTS : -

The DICOM path is the more complete part of the implementation. It performs metadata de-identification, deterministic pseudonymisation and UID regeneration, checks pixel-level rules and then stores the sanitized file and metadata. The main batch function also isolates failures per file instead of stopping the whole run when one DICOM causes an exception.

The PDF path follows a similar sequence, but the actual de-identification mechanism is different because the PHI is visible in document text rather than DICOM tags. The parser locates fields using their labels and word coordinates, HMACs the patient ID, redacts the patient name and ID, clears the PDF metadata and then stores the resulting document.

One design choice I think is important here is that the pseudonymisation happens before the value is used for storage or database identity. The patient HMAC is deterministic, so the same raw patient ID maps to the same pseudonymous value when the same secret is used. The DICOM UIDs are treated in the same way, but with a namespace for study, series, SOP and frame values.

The metadata layer also gives the pipeline somewhere to record what happened to a file instead of making the object store the only source of truth. This matters for failed or reviewed pixel cases, since the stored record can retain the processing state without retaining the original PHI.

There are still a few operational gaps. The default main execution currently enables only the DICOM branch, and the current PDF filename uses an in-run sequence for repeated reports from a pseudonymous patient. I would tighten the PDF naming for reruns if this were moved directly into a larger production workload.

KEY FINDINGS: -

- The pipeline is split by file type, but storage, metadata and logging remain shared. This keeps DICOM-specific and PDF-specific logic from leaking into the rest of the system.
- Patient IDs are pseudonymised with HMAC, while DICOM study/series/SOP/frame identifiers are regenerated deterministically. This gives stable identifiers without retaining the original values.
- DICOM pixel data is treated as a separate PHI surface. The pipeline distinguishes cleanable graylist data, whitelist matches, blacklist matches and review cases instead of treating every detected hit the same way.
- Sanitized objects go to MinIO and extracted metadata goes to SQLite, so the database does not need to hold the actual healthcare files.
- The implementation already has per-file exception handling and batch counts, which is enough to see where a run failed. For larger deployment, the next step would be richer metrics rather than changing the whole architecture.

FUTURE WORK: -

- Add a small **on-premise NLP model** for cases where the current rule-based PDF extraction is not enough. The model would identify and tag possible PII fields in the document, with a confidence threshold set around **90%** initially. High-confidence detections can be redacted automatically, while anything below the threshold would be marked for manual review instead of being silently deleted. The threshold should then be tuned against a labelled validation set before using the system in production.
- Add a small CI stage for de-identification regression tests, especially around DICOM UID replacement, PDF redaction and pixel PHI rules. These are the parts where a silent behaviour change is more dangerous than a normal code regression.
- Modify the codebase to be concurrent, as this is a very high ROI for speed and latency reduction. The DICOM files are processed independently from each other, so there is no reason for the pipeline to wait for one file to finish before starting the next one. A bounded worker pool could process multiple files in parallel, particularly around file reading, de-identification and MinIO uploads. The metadata writes would need to be handled separately since the current SQLite connection should not simply be shared across workers. Instead, workers can return their processing records and the main process can commit them once the batch finishes.